

Материал за подготовка

Важно: Този материал обхваща всички теми, които ще бъдат проверени в контролната работа. Прочети внимателно всички раздели и упражнявай с примерите!

1. Основни понятия за бази данни

1.1. Какво е база данни?

База данни е организирана колекция от свързани данни, съхранявани електронно. Тя позволява лесно търсене, сортиране, филтриране и обработка на информация.

Защо използваме бази данни вместо обикновени файлове?

- **Организация:** Данните са структурирани в таблици, което ги прави лесни за намиране и използване
- **Бързо търсене:** Можем бързо да намерим конкретна информация (напр. всички ученици от клас "11a")
- **Избягване на дублиране:** Данните се съхраняват на едно място, което предотвратява повторения
- **Връзки между данни:** Можем да свързваме данни от различни таблици (напр. ученици с техните класове)
- **Цялост на данните:** Можем да гарантираме, че данните са коректни и пълни

Примери за бази данни в реалния живот:

- **Училищна система:** Списък с ученици, техните класове, оценки, предмети
- **Библиотека:** Каталог с книги, автори, заеми, читатели
- **Магазин:** Регистър на клиенти, продукти, продажби, доставки
- **Болница:** Пациенти, лекари, прегледи, лекарства
- **Банка:** Клиенти, сметки, транзакции, кредити

Как работи базата данни: Представи си, че имаш огромна шкаф с много чекмеджета. Всяко чекмедже (таблица) съдържа определени документи (записи), които са организирани по категории (полета). Когато търсиш нещо, базата данни бързо намира точното чекмедже и документа, от който се нуждаеш.

1.2. Система за управление на бази данни (СУБД)

СУБД (Система за Управление на Базы Данни) е програма, чрез която създаваме, поддържаме и използваме бази данни. Тя е инструментът, който ни позволява да работим с данните.

Какво прави СУБД?

- **Създаване:** Помага ни да създадем структурата на базата данни (таблицы, полета, релации)
- **Въвеждане:** Позволява ни да добавяме, променяме и изтриваме данни
- **Търсене:** Позволява ни да търсим и филтрираме данни по различни критерии
- **Защита:** Осигурява сигурност и контрол на достъпа до данните
- **Резервни копия:** Позволява ни да правим резервни копия на данните

Примери за СУБД:

- **Microsoft Access** – за малки и средни бази данни (до няколко хиляди записи). Лесна за използване, подходяща за начинаещи. Използваме я в училище.
- **MySQL, PostgreSQL** – за по-големи проекти (милиони записи). Използват се за уебсайтове и приложения.
- **SQL Server, Oracle** – за корпоративни системи. Много мощни, но сложни за използване.

Защо използваме Microsoft Access в училище? Access е лесен за научаване, има визуален интерфейс (можем да виждаме таблиците и да работим с тях графично), и е достатъчно мощен за нашите нужди. Освен това е част от Microsoft Office пакета, който вече познаваме.

1.3. Структура на базата данни

Базата данни се състои от **таблици**, всяка таблица съдържа **записи** (редове) и **полета** (колони).

Аналогия с Excel: Ако познаваш Excel, можеш да мислиш за базата данни като за работна книга (workbook), таблицата като за лист (sheet), записите като за редове, а полетата като за колони. Но има важни разлики:

- В базата данни данните са **структурирани** – всяко поле има определен тип данни
- Можем да създаваме **връзки** между таблиците
- Можем да гарантираме **цялост на данните** (напр. не може да има ученик без клас)

Пример: Таблица „Ученици“

ИД	Име	Клас	Успех
1	Иван Петров	11a	5.50
2	Мария Георгиева	11a	6.00
3	Петър Стоянов	11б	5.25

Подробно обяснение на структурата:

- **Таблица „Ученици“** – това е цялата таблица. Името на таблицата описва какъв тип обекти съхраняваме (в случая – ученици). Всяка таблица в базата данни съхранява информация за един тип обекти.
- **Поле (field)** – това е една колона в таблицата. Всяко поле съхранява определен вид информация:
 - Полето „ИД“ съхранява уникален номер за всеки ученик
 - Полето „Име“ съхранява имената на учениците
 - Полето „Клас“ съхранява в кой клас е всеки ученик
 - Полето „Успех“ съхранява средния успех на всеки ученик
- **Запис (record)** – това е един ред в таблицата. Всеки запис съдържа всички данни за един конкретен обект:
 - Първият запис (ред 1) съдържа данните за Иван Петров: ИД=1, Име="Иван Петров", Клас="11a", Успех=5.50
 - Вторият запис (ред 2) съдържа данните за Мария Георгиева: ИД=2, Име="Мария Георгиева", Клас="11a", Успех=6.00

Всеки запис представлява един конкретен ученик с всичките му данни.

Визуализация: Представи си таблицата като реална таблица в училище. Колоните (полетата) са категориите информация (ИД, Име, Клас, Успех), а редовете (записите) са отделните ученици, които седят на всеки ред.

2. Типове данни в MS Access

При създаване на таблица трябва да изберем подходящ тип данни за всяко поле. **Типът данни** определя какъв вид информация може да съдържа полето и как може да се използва.

Защо е важно да изберем правилния тип данни?

- **Точност:** Правилният тип гарантира, че данните са коректни (напр. датата е наистина дата, а не текст)
- **Изчисления:** Някои типове позволяват математически операции (числа), други не (текст)
- **Сортиране:** Различните типове се сортират различно (датите хронологически, числата числено, текстът азбучно)
- **Проверка:** Access може автоматично да проверява дали въведените данни са валидни за този тип
- **Ефективност:** Правилният тип заема по-малко място и работи по-бързо

Тип данни	Описание	Кога се използва	Примери
Кратък текст	Текст до 255 символа. Може да съдържа букви, цифри, символи, интервали.	Имена, адреси, телефонни номера, кодове, категории	„Иван Петров“, „0888123456“, „ул. Витоша 15“, „Електроника“
Дълъг текст	Текст до 65 535 символа	Описания, бележки	Описание на продукт
Число	Цели числа	Бройки, възраст, количество	15, 100, 2500
Десетично число	Числа с десетична запетая	Цени, успех, проценти	19.99, 5.50, 3.14
Дата/час	Дата и/или час	Дата на раждане, дата на заемане	15.03.2008, 01.01.2024
Автоматично число (AutoNumber)	Автоматично генерирано число	Първичен ключ	1, 2, 3, 4... (автоматично)

Да/Не (Boolean)	Логическа стойност	Проверки, статуси	Да/Не, True/False
--------------------	-----------------------	----------------------	-------------------

Важно за избора на тип данни - Подробни обяснения:

1. Телефонен номер → Кратък текст

- **Защо не "Число"?** Телефонните номера могат да започват с 0 (0888123456). Ако използваме тип "Число", Access ще премахне водещата нула и ще получим 888123456, което е грешно.
- **Допълнителни причини:** Телефонните номера може да имат тирета, скоби, интервали (0888-123-456, (0888) 123 456), които не са валидни за числа. Освен това не правим математически изчисления с телефонни номера (не ги събираме, изваждаме и т.н.), така че няма нужда от числов тип.
- **Пример:** "0888123456" като текст запазва водещата нула и може да съдържа форматиране.

2. Дата на раждане → Дата/час

- **Защо не "Кратък текст"?** Ако използваме текст, не можем да правим изчисления (напр. колко години е човекът = днешна дата - дата на раждане). Не можем да сортираме хронологически (текстът се сортира азбучно: "01.01.2020" идва преди "15.03.2010", което е грешно). Не можем да проверяваме дали датата е валидна (напр. "32.13.2020" би било невалидна дата, но като текст е валиден).
- **Предимства на "Дата/час":** Access автоматично проверява дали датата е коректна, ни позволява да изчисляваме разлики между дати, да сортираме хронологически, и да филтрираме по период (напр. всички родени след 2010 г.).
- **Пример:** С тип "Дата/час" можем да напишем заявка: "Покажи всички ученици, които са на 18 или повече години" – Access автоматично изчислява възрастта.

3. Успех → Десетично число

- **Защо не "Число"?** Успехът е десетично число (5.50, 4.75, 6.00). Типът "Число" в Access може да бъде само цели числа (5, 4, 6), без десетична част.
- **Защо не "Кратък текст"?** Ако използваме текст, не можем да правим математически изчисления (среден успех, сума, сравнение). Текстът "5.50" и "6.00" при сортиране азбучно ще даде "5.50" преди "6.00", но "6.00" е по-висок успех.
- **Предимства на "Десетично число":** Можем да изчисляваме среден успех, да сравняваме успехи, да сортираме числено (5.25 < 5.50 < 6.00).

- **Пример:** С десетично число можем да напишем заявка: "Покажи всички ученици с успех над 5.50" – Access прави числово сравнение.

4. Первичен ключ → Автоматично число

- **Защо не "Число"?** Ако използваме "Число", трябва сами да задаваме стойността за всеки нов запис. Рискът е да забравим, да зададем същата стойност два пъти (което нарушава уникалността), или да зададем грешна стойност.
- **Предимства на "Автоматично число":** Access автоматично генерира уникални числа (1, 2, 3, 4...). Не трябва да се притесняваме да забравим да зададем стойност или да създадем дубликат. Access сам се грижи за уникалността.
- **Допълнително:** Автоматичното число винаги е уникално, никога не е празно, и не може да се променя (което е важно за первичен ключ).
- **Пример:** Когато добавяме нов ученик, Access автоматично задава ИД = 5 (ако последният ученик е с ИД = 4). Ние не трябва да мислим за това.

3. Ключове в базата данни

3.1. Первичен ключ (Primary Key)

Первичен ключ е поле (или комбинация от полета), което **еднозначно идентифицира всеки запис** в таблицата. Това означава, че използвайки стойността на первичния ключ, можем да намерим точно един запис в таблицата.

Аналогия: Представи си, че первичният ключ е като ЕГН (Единен граждански номер) на човек. Всеки човек има уникално ЕГН, и по ЕГН можем да намерим точно този човек. По същия начин, всеки запис в таблицата има уникална стойност в полето на первичния ключ, и по тази стойност можем да намерим точно този запис.

Свойства на първичния ключ - Подробно обяснение:

- **Уникален** – всяка стойност трябва да е различна за всеки запис. Това означава, че не може да има два записа с еднаква стойност в полето на първичния ключ. Например, ако имаме двама ученика, единият не може да има ИД = 1, а другият също ИД = 1. Всеки трябва да има различен ИД (1, 2, 3...). **Защо е важно?** Ако два записа имат еднаква стойност в първичния ключ, Access няма да знае кой запис да избере, когато търсим по този ключ.
- **Не може да е празен (NOT NULL)** – всеки запис трябва да има стойност в полето на първичния ключ. Не може да има запис с празен (NULL) първичен ключ. **Защо е важно?** Ако първичният ключ е празен, не можем да идентифицираме записа. Това е като човек без ЕГН – не знаем кой е той.
- **Обикновено се използва AutoNumber** – Access автоматично генерира уникални числа (1, 2, 3, 4...) за всеки нов запис. **Защо е удобно?** Не трябва сами да мислим за уникалността – Access се грижи за това. Освен това AutoNumber винаги е уникален и никога не е празен, което отговаря на всички изисквания за първичен ключ.

Пример: В таблица „Ученици“ полето „ИД“ е първичен ключ:

ИД (РК)	Име	Клас
1	Иван Петров	11а
2	Мария Георгиева	11а

Всяко ИД е уникално и идентифицира точно един ученик.

3.2. Външен ключ (Foreign Key)

Външен ключ е поле в една таблица, което препраща към първичен ключ в друга таблица. Той създава връзка (релация) между таблиците.

Пример: Таблица „Ученици“ има поле „ИД_клас“, което е външен ключ към таблица „Класове“:

Таблица „Класове“:

ИД_клас (PK)	Име
1	11a
2	11б

Таблица „Ученици“:

ИД (PK)	Име	ИД_клас (FK)
1	Иван Петров	1
2	Мария Георгиева	1

Поле то „ИД_клас“ в „Ученици“ е външен ключ, който сочи към „ИД_клас“ в „Класове“.

4. Релации между таблици

Релациите определят как таблиците са свързани една с друга.

4.1. Релация „Един към много“ (1:M)

Един запис в едната таблица съответства на **много записи** в другата таблица.

Пример: Един клас има много ученици

Как да разберем коя таблица е "една" и коя е "много":

- Таблица „Класове“ е "една" (1), защото има **един запис** за всеки клас (напр. един запис за клас "11а")
- Таблица „Ученици“ е "много" (М), защото има **много записи** за ученици от един и същи клас (напр. 25 ученика от клас "11а" = 25 записа)

Визуализация:

Таблица „Класове“ (1 - една)	Таблица „Ученици“ (М - много)
ИД_клас: 1 Име: "11а"	ИД: 1, Име: "Иван", ИД_клас: 1 ИД: 2, Име: "Мария", ИД_клас: 1 ИД: 3, Име: "Петър", ИД_клас: 1 ... (още 22 ученика)

Един клас (1) → Много ученици (М)

Други примери:

- **Една книга (1) → много заеми (М)** – една книга може да бъде заета много пъти
- **Един продукт (1) → много продажби (М)** – един продукт може да бъде продаван много пъти
- **Един член (1) → много абонаменти (М)** – един член може да има много абонаменти през времето

4.2. Релация „Много към много“ (М:М)

Много записи в едната таблица съответстват на **много записи** в другата таблица.

Пример: Ученици и предмети**Как работи:**

- **Един ученик** учи **много предмети** (напр. Иван учи Математика, Физика, Химия)
- **Един предмет** се изучава от **много ученици** (напр. Математика се изучава от Иван, Мария, Петър и др.)

Визуализация:

Ученик	Предмети, които учи
Иван	Математика, Физика, Химия, История
Мария	Математика, Физика, Биология, Литература

Иван учи много предмети И Математика се изучава от много ученици = M:M

Забележка: За реализиране на M:M релация в база данни се използва **междинна таблица** (junction table), която свързва двете таблици.

4.3. Referential Integrity (Референтна цялост)

Referential Integrity гарантира, че:

- Не може да се добави запис с външен ключ, който **не съществува** в родителската таблица
- Не може да се изтрие запис от родителската таблица, ако има свързани записи в дъщерната таблица

Пример: Ако имаме релация между „Класове" и „Ученици":

- **✗ Не може** да добавим ученик с ИД_клас = 99, ако няма клас с ИД = 99
- **✗ Не може** да изтрием клас „11a", ако има ученици, които са в този клас

5. Обекти в MS Access

Обект	Назначение	Кога се използва
-------	------------	------------------

Таблица (Table)	Съхранява данните	Основна структура за данни
Форма (Form)	Въвеждане и визуализиране на данни	За удобно въвеждане и преглед на данни от потребителя
Заявка (Query)	Извличане и обработка на данни	За търсене, филтриране, сортиране, изчисления
Отчет (Report)	Форматирано представяне и печат	За печат на данни с групиране, сумиране, стилизиране

6. SQL заявки - Основни команди

6.1. SELECT - Извличане на данни

SELECT се използва за извличане (четене) на данни от таблица. Това е най-често използваната SQL команда. Тя **не променя** данните в таблицата, а само ги чете и показва.

Как работи SELECT:

1. Казваме на Access **какви полета** искаме да видим (SELECT колона1, колона2...)
2. Казваме **от коя таблица** да вземе данните (FROM име_на_таблица)
3. Опционално задаваме **условия** за филтриране (WHERE условие)
4. Access връща резултата като нова таблица с избраните данни

Синтаксис:

```
SELECT колона1, колона2, ...  
FROM име_на_таблица  
WHERE условие;
```

Подробни примери с обяснения:**Пример 1: Извежда всички полета от всички записи**

```
SELECT * FROM Ученици;
```

Обяснение: Символът * означава "всички полета". Тази заявка ще изведе всички колони (ИД, Име, Клас, Успех) и всички редове от таблицата "Ученици".

Пример 2: Извежда само имената на всички ученици

```
SELECT Име FROM Ученици;
```

Обяснение: Тук избираме само полето "Име". Резултатът ще бъде списък само с имената на всички ученици, без другите данни.

Пример 3: Извежда имената и успеха на учениците

```
SELECT Име, Успех FROM Ученици;
```

Обяснение: Избираме две колони: "Име" и "Успех". Резултатът ще бъде таблица с две колони – имената и успехите на всички ученици.

Важно: Имената на полетата се пишат точно както са в таблицата. Ако полето се казва "Име", трябва да напишем точно "Име", а не "име" или "ИМЕ" (в някои СУБД това е важно).

6.2. WHERE - Филтриране на данни

WHERE задава условие за филтриране – показва само редовете, които отговарят на условието. Без WHERE заявката ще върне всички записи от таблицата.

Как работи WHERE:

1. Access преглежда всеки запис в таблицата
2. Проверява дали записът отговаря на условието след WHERE
3. Ако условието е вярно, записът се включва в резултата
4. Ако условието е невярно, записът се пропуска

Важно: Текстовите стойности в условията се ограждат с единични кавички ('11a'), а числовите стойности не се ограждат (5.00).

Подробни примери с обяснения:

Пример 1: Ученици от клас "11a"

```
SELECT * FROM Ученици WHERE Клас = '11a';
```

Обяснение: Условието `Клас = '11a'` означава "покажи само записите, където полето Клас е равно на '11a'". Access ще прегледа всички ученици и ще покаже само тези, които са в клас "11a". Забележи, че '11a' е в единични кавички, защото е текст.

Пример 2: Ученици с успех над 5.00

```
SELECT * FROM Ученици WHERE Успех > 5.00;
```

Обяснение: Условието `Успех > 5.00` означава "покажи само записите, където полето Успех е по-голямо от 5.00". Access ще покаже само учениците с успех над 5.00 (напр. 5.25, 5.50, 6.00), но няма да покаже тези с успех 5.00 или по-нисък. Забележи, че 5.00 не е в кавички, защото е число.

Пример 3: Ученици с успех поне 4.50 И от клас "11б"

```
SELECT * FROM Ученици  
WHERE Успех >= 4.50 AND Клас = '11б';
```

Обяснение: Тук имаме две условия, свързани с `AND` (И). Това означава, че и двете условия трябва да са верни едновременно:

- Успехът трябва да е поне 4.50 (`>=` означава "по-голямо или равно")
- И класът трябва да е "11б"

Access ще покаже само учениците, които са И в клас "11б" И имат успех поне 4.50. Ако ученик е в клас "11б", но има успех 4.00, няма да бъде показан. Ако ученик има успех 5.00, но е в клас "11а", също няма да бъде показан.

Пример 4: Ученици с успех над 5.50 ИЛИ от клас "12а"

```
SELECT * FROM Ученици  
WHERE Успех > 5.50 OR Клас = '12а';
```

Обяснение: Тук условията са свързани с `OR` (ИЛИ). Това означава, че поне едно от условията трябва да е вярно:

- Успехът е над 5.50, ИЛИ
- Класът е "12а"

Access ще покаже всички ученици, които имат успех над 5.50 (независимо от класа), И също всички ученици от клас "12а" (независимо от успеха). Ако един ученик е в клас "12а" И има успех над 5.50, той също ще бъде показан (защото отговаря на поне едно условие).

Оператори за сравнение:

- **=** – равно
- **!=** или **<>** – различно
- **>** – по-голямо
- **<** – по-малко
- **>=** – по-голямо или равно
- **<=** – по-малко или равно
- **AND** – логическо И (и двете условия трябва да са верни)
- **OR** – логическо ИЛИ (поне едно условие трябва да е вярно)

6.3. ORDER BY - Сортиране

ORDER BY подрежда резултатите по определено поле. Без **ORDER BY** записите се показват в произволен ред (обикновено в реда, в който са добавени).

Как работи ORDER BY:

- **ASC** (Ascending) – възходящ ред: от най-малкото към най-голямото (за числа: 1, 2, 3...; за текст: А, Б, В...; за дати: по-стари към по-нови). **ASC** е по подразбиране, така че не е задължително да го пишем.
- **DESC** (Descending) – низходящ ред: от най-голямото към най-малкото (за числа: 3, 2, 1...; за текст: В, Б, А...; за дати: по-нови към по-стари).
- Можем да сортираме по няколко полета – първо по едно поле, а при еднакви стойности – по второ поле.

Подробни примери с обяснения:

Пример 1: Сортиране по успех във възходящ ред

```
SELECT Име, Успех FROM Ученици ORDER BY Успех;
```

Обяснение: Заявката ще подреди учениците по успех от най-нисък към най-висок (напр. 4.00, 4.50, 5.00, 5.50, 6.00). Тъй като не сме написали ASC или DESC, Access използва ASC (възходящ ред) по подразбиране.

Пример 2: Сортиране по успех в низходящ ред

```
SELECT Име, Успех FROM Ученици ORDER BY Успех DESC;
```

Обяснение: Заявката ще подреди учениците по успех от най-висок към най-нисък (напр. 6.00, 5.50, 5.00, 4.50, 4.00). Това е полезно, когато искаме да видим най-добрите ученици първо.

Пример 3: Сортиране по няколко полета

```
SELECT Име, Клас, Успех FROM Ученици  
ORDER BY Клас ASC, Успех DESC;
```

Обяснение: Тук сортираме по две полета:

1. **Първо по Клас** (ASC – възходящо): Учениците ще бъдат групирани по класове (напр. първо всички от "11a", после от "11б", после от "12a")
2. **После по Успех** (DESC – низходящо): В рамките на всеки клас, учениците ще бъдат подредени по успех от най-висок към най-нисък

Резултатът ще изглежда така:

- Клас "11a": ученик с успех 6.00, после 5.50, после 5.00...
- Клас "11б": ученик с успех 5.75, после 5.25, после 4.50...
- И т.н.

Това е полезно, когато искаме да видим най-добрите ученици от всеки клас.

6.4. GROUP BY - Групиране

GROUP BY групира записите по общи признаци и позволява изчисления върху групите.

Примери:

```
-- Среден успех по класове
SELECT Клас, AVG(Успех) FROM Ученици GROUP BY Клас;

-- Обща сума на продажбите по категория
SELECT Категория, SUM(Цена) FROM Продукти GROUP BY Категория;

-- Брой ученици по клас
SELECT Клас, COUNT(*) FROM Ученици GROUP BY Клас;
```

Агрегатни функции (използват се с GROUP BY):

- **COUNT(*)** – преброява броя на записите
- **AVG(поле)** – изчислява средната стойност
- **SUM(поле)** – сумира стойностите
- **MAX(поле)** – намира максималната стойност
- **MIN(поле)** – намира минималната стойност

6.5. INSERT INTO - Добавяне на записи

INSERT INTO се използва за добавяне на нов запис в таблица.

```
-- Добавяне на нов ученик
INSERT INTO Ученици (Име, Клас, Успех)
VALUES ('Петър Иванов', '11а', 5.75);

-- Добавяне на нов продукт
INSERT INTO Продукти (Име, Категория, Цена)
VALUES ('Лаптоп', 'Електроника', 1200.00);
```

6.6. UPDATE - Промяна на данни

UPDATE се използва за промяна (актуализация) на стойности в съществуващи записи.

```
-- Промяна на успеха на всички ученици от клас "11a"  
UPDATE Ученици  
SET Успех = 5.50  
WHERE Клас = '11a';  
  
-- Промяна на цената на конкретен продукт  
UPDATE Продукти  
SET Цена = 1500.00  
WHERE ИД = 5;
```

6.7. DELETE - Изтриване на записи

DELETE се използва за изтриване на записи от таблица.

Внимание: DELETE изтрива записите завинаги! Винаги използвай WHERE, за да не изтриеш всички записи!

```
-- Изтриване на ученик с конкретно ИД  
DELETE FROM Ученици WHERE ИД = 10;  
  
-- Изтриване на всички продукти от категория "Стар"  
DELETE FROM Продукти WHERE Категория = 'Стар';
```

7. Моделиране на база данни - Стъпки

Стъпки при проектиране на база данни:

1. **Определи обектите** – какви неща трябва да съхраняваме? (напр. ученици, класове, книги, заеми)
2. **Създай таблици** – всяка таблица представлява един тип обект
3. **Определи полетата** – каква информация трябва да съхраняваме за всеки обект?
4. **Избери типове данни** – подходящ тип за всяко поле
5. **Определи първичните ключове** – кое поле уникално идентифицира всеки запис?
6. **Създай релации** – как таблиците са свързани? (1:M, M:M)
7. **Добави външни ключове** – полета, които сочат към други таблици
8. **Включи Referential Integrity** – за да гарантираш цялостта на данните

7.1. Пример: База данни „Училище“

Таблица „Класове“:

Поле	Тип данни	Описание
ИД_клас	AutoNumber	Първичен ключ
Име	Кратък текст	Напр. "11а", "11б"
КласенРъководител	Кратък текст	Име на класния ръководител

Таблица „Ученици“:

Поле	Тип данни	Описание
ИД_ученик	AutoNumber	Първичен ключ
Име	Кратък текст	Първо име
Фамилия	Кратък текст	Фамилия
ИД_клас	Число	Външен ключ към „Класове“
Успех	Десетично число	Среден успех

Релация: 1:М между „Класове“ (1) и „Ученици“ (М) – един клас има много ученици.

7.2. Пример: База данни „Библиотека“

Таблица „Книги“:

- ИД_книга (AutoNumber, PK)
- Заглавие (Кратък текст)
- Автор (Кратък текст)
- Година (Число)
- НаличниБройки (Число)

Таблица „Заеми“:

- ИД_заем (AutoNumber, PK)
- ИД_книга (Число, FK → Книги)
- Ученик (Кратък текст)
- ДатаНаЗаемане (Дата/час)
- ДатаНаВръщане (Дата/час)

Релация: 1:М между „Книги“ и „Заеми“ – една книга може да бъде заета много пъти.

7.3. Пример: База данни „Магазин“**Таблица „Продукти“:**

- ИД_продукт (AutoNumber, PK)
- Име (Кратък текст)
- Категория (Кратък текст)
- Цена (Десетично число)
- Наложение (Число)

Таблица „Продажби“:

- ИД_продажба (AutoNumber, PK)
- ИД_продукт (Число, FK → Продукти)
- Количество (Число)
- Дата (Дата/час)
- ОбщаСума (Десетично число)

Релация: 1:М между „Продукти“ и „Продажби“ – един продукт може да бъде продаван много пъти.

7.4. Пример: База данни „Фитнес клуб“

Таблица „Членове“:

- ИД_член (AutoNumber, PK)
- Име (Кратък текст)
- Телефон (Кратък текст)
- ДатаНаРегистрация (Дата/час)

Таблица „Абонаменти“:

- ИД_абонамент (AutoNumber, PK)
- ИД_член (Число, FK → Членове)
- ТипАбонамент (Кратък текст)
- НачалнаДата (Дата/час)
- КрайнаДата (Дата/час)
- Цена (Десетично число)

Релация: 1:М между „Членове“ и „Абонаменти“ – един член може да има много абонаменти през времето.

8. Често срещани задачи и решения

8.1. Извеждане на данни с условия

Задача: Изведи всички ученици от клас "11a" с успех над 5.00

```
SELECT * FROM Ученици  
WHERE Клас = '11a' AND Успех > 5.00;
```

8.2. Сортиране на резултатите

Задача: Изведи имената и успеха на учениците, подредени по успех в низходящ ред

```
SELECT Име, Успех FROM Ученици  
ORDER BY Успех DESC;
```

8.3. Групиране и изчисления

Задача: Намери средния успех по класове

```
SELECT Клас, AVG(Успех) FROM Ученици  
GROUP BY Клас;
```

8.4. Преброяване на записи

Задача: Колко ученика има в клас "116"?

```
SELECT COUNT(*) FROM Ученици  
WHERE Клас = '116';
```

Обяснение: `COUNT(*)` преброява броя на редовете, които отговарят на условието. Резултатът е число (напр. 25 ученика).

9. Ключови думи за запомняне

Основни понятия:

- **База данни** – организирана колекция от свързани данни
- **СУБД** – система за управление на бази данни (напр. MS Access)
- **Таблица** – колекция от записи, организирани в редове и колони
- **Поле** – една колона в таблицата
- **Запис** – един ред в таблицата
- **Първичен ключ** – поле, което уникално идентифицира всеки запис
- **Външен ключ** – поле, което препраща към първичен ключ в друга таблица
- **Релация 1:М** – един запис в едната таблица съответства на много в другата
- **Referential Integrity** – гаранция за цялостта на данните

SQL команди:

- **SELECT** – извличане на данни
- **WHERE** – филтриране
- **ORDER BY** – сортиране
- **GROUP BY** – групиране
- **INSERT INTO** – добавяне на записи
- **UPDATE** – промяна на данни
- **DELETE** – изтриване на записи

10. Съвети за успешно решаване на контролната работа

1. **Прочети внимателно всеки въпрос** – не бързай, разбери какво се иска.
2. **При тестовите въпроси** – прегледай всички отговори преди да избереш.
3. **При SQL заявки** – започни с основните части: SELECT, FROM, WHERE, ORDER BY.
4. **При моделиране на база данни** – помисли какви обекти имаш, какви полета са нужни, какви релации има.
5. **Провери типовете данни** – дали са подходящи за всяко поле.
6. **Не забравяй първичните ключове** – всяка таблица трябва да има първичен ключ.
7. **Помни за Referential Integrity** – когато имаш релации, трябва да включиш тази опция.

Успех на контролната работа! 🎓

Прочети този материал внимателно, упражнявай с примерите и запомни ключовите понятия. Ако разбираш всичко тук, ще се справиш отлично!

11. ВАЖНИ SQL ЗАЯВКИ ЗА УПРАЖНЕНИЕ

⚠ ВНИМАНИЕ! Обърни специално внимание на тези SQL заявки!

Проучи ги внимателно, разбери какво прави всяка една, и упражнявай да ги пишеш наизуст!

11.1. Основни SQL заявки

Заявка 1: Извеждане на всички ученици от определен клас

```
SELECT * FROM Ученици WHERE Клас = '11a';
```

Какво прави: Извежда всички полета (*) на всички ученици, които са в клас "11a".

Важно: Текстът '11a' е в единични кавички, защото е текстова стойност.

Заявка 2: Извеждане с сортиране

```
SELECT Име, Успех FROM Ученици ORDER BY Успех DESC;
```

Какво прави: Извежда само имената и успеха на учениците, подредени по успех в низходящ ред (от най-висок към най-нисък).

Важно: DESC означава низходящ ред. Ако искаме възходящ, пишем ASC или не пишем нищо.

Заявка 3: Филтриране с множество условия (AND)

```
SELECT Име FROM Ученици WHERE Успех >= 4.50 AND Клас = '11б';
```

Какво прави: Извежда само имената на учениците, които имат успех поне 4.50 И са в клас "11б".

Важно: AND означава, че и двете условия трябва да са верни едновременно.

Заявка 4: Извеждане на конкретни полета с условие

```
SELECT Име, Успех FROM Ученици WHERE Успех > 5.00;
```

Какво прави: Извежда само имената и успеха на учениците с успех над 5.00 (не включва 5.00).

Важно: > означава "по-голямо", а >= означава "по-голямо или равно".

Заявка 5: Извеждане с условие и сортиране

```
SELECT Име, Успех FROM Ученици  
WHERE Успех >= 5.50  
ORDER BY Успех DESC;
```

Какво прави: Извежда имената и успеха на учениците с успех поне 5.50, подредени по успех в низходящ ред.

Важно: Обърни внимание на реда: първо WHERE (филтриране), после ORDER BY (сортиране).

Заявка 6: Сортиране по няколко полета

```
SELECT * FROM Книги ORDER BY Година ASC, Заглавие;
```

Какво прави: Извежда всички книги, подредени първо по година във възходящ ред, а при еднаква година – по заглавие.

Важно: Когато сортираме по няколко полета, първото поле е приоритетно. При еднакви стойности в първото поле, се сортира по второто поле.

Заявка 7: Преброяване на записи (COUNT)

```
SELECT COUNT(*) FROM Ученици WHERE Клас = '11б';
```

Какво прави: Преброява колко ученика има в клас "11б". Връща число (напр. 25).

Важно: COUNT(*) преброява броя на редовете, които отговарят на условието. Резултатът е едно число, не списък с ученици.

Заявка 8: Филтриране с AND

```
SELECT * FROM Продукти  
WHERE Категория = 'Електроника' AND Цена < 500;
```

Какво прави: Извежда всички продукти от категория "Електроника" с цена под 500 лв.

Важно: И двете условия трябва да са верни: категорията трябва да е "Електроника" И цената трябва да е под 500.

Заявка 9: Сортиране по цена

```
SELECT Име FROM Продукти ORDER BY Цена DESC;
```

Какво прави: Извежда имената на продуктите, подредени по цена в низходящ ред (от най-скъпите към най-евтините).

11.2. Заявки с групиране и изчисления

Заявка 10: Групиране и изчисления (AVG)

```
SELECT Клас, AVG(Успех) FROM Ученици GROUP BY Клас;
```

Какво прави: Извежда всеки клас и средния успех на учениците в този клас. Групира резултатите по клас.

Важно: AVG() изчислява средната стойност. GROUP BY групира записите по определено поле. В SELECT можем да използваме само полета, които са в GROUP BY, или агрегатни функции (AVG, SUM, COUNT, и т.н.).

Заявка 11: Групиране и сумиране (SUM)

```
SELECT Категория, SUM(Цена) FROM Продукти GROUP BY Категория;
```

Какво прави: Извежда всяка категория и общата сума на цените на всички продукти в тази категория.

Важно: SUM() сумира стойностите. GROUP BY групира записите по категория, и за всяка категория изчислява сумата на цените.

11.3. Обобщение на SQL командите

Команди, които ще срещнеш:

- **SELECT** – извличане на данни (най-често използвана)
- **FROM** – указва от коя таблица да се вземат данните
- **WHERE** – филтриране на данни по условие
- **ORDER BY** – сортиране на резултатите
- **GROUP BY** – групиране на записите
- **AVG()** – изчисляване на средна стойност
- **SUM()** – сумиране на стойности
- **COUNT()** – преброяване на записи

Стъпки за решаване на SQL задачи:

1. **Прочети внимателно** какво се иска – какви данни трябва да изведем?
2. **Определи кои полета** трябва да изведем (SELECT колона1, колона2... или SELECT *)
3. **Определи от коя таблица** да вземем данните (FROM име_на_таблица)
4. **Провери дали има условие** за филтриране (WHERE условие)
5. **Провери дали трябва сортиране** (ORDER BY поле ASC/DESC)
6. **Провери дали трябва групиране** (GROUP BY поле) – обикновено се използва с агрегатни функции (AVG, SUM, COUNT)
7. **Провери синтаксиса** – всички текстове в единични кавички, правилни оператори (=, >, <, >=, <=, AND, OR)

Често срещани грешки, които трябва да избягваш:

- **✗** Забравяне на единичните кавички около текстовите стойности:
`WHERE Клас = 11a` (грешно) → `WHERE Клас = '11a'` (правилно)
- **✗** Забравяне на WHERE при филтриране: `SELECT * FROM Ученици Клас = '11a'` (грешно) → `SELECT * FROM Ученици WHERE Клас = '11a'` (правилно)
- **✗** Грешен ред на командите: ORDER BY преди WHERE (грешно) → WHERE преди ORDER BY (правилно)
- **✗** Забравяне на GROUP BY при използване на агрегатни функции:
`SELECT Клас, AVG(Успех) FROM Ученици` (грешно) → `SELECT Клас, AVG(Успех) FROM Ученици GROUP BY Клас` (правилно)
- **✗** Объркване на AND и OR: AND означава "и двете условия", OR означава "поне едно условие"